INSTITUTO FEDERAL
Santa Catarina

Programação e Engenharia de Software

Unidade 8 – Programação Orientada a Objetos

Prof. Thiago Waltrik

INSTITUTO FEDERAL
Santa Catarina

Conhecimentos: Unidade 8

- Visão geral sobre paradigmas de programação;
- Introdução à Programação Orientada a Objetos;
- Classes e objetos;
- Atributos e métodos;
- Construtores e destrutores de objetos;
- Modificadores de acesso e visibilidade de membros das classes;

INSTITUTO FEDERAL
Santa Catarina

Objetivos

Conhecer os principais conceitos da Programação Orientada a Objetos (POO) e aplicar técnicas de POO em Python.



INSTITUTO FEDERAL
Santa Catarina

Visão geral sobre paradigmas de programação

Programação Estruturada (PE)

- É o paradigma de programação que estudamos até agora no curso. Começou a ganhar popularidade no início dos anos de 1970.
- O foco está nas funções e no fluxo sequencial de um programa.
- Funciona bem para programas/aplicações pequenos(as).
- Exemplos de linguagens de programação estruturada: C (1972), Pascal (1970), Fortran e BASIC.

INSTITUTO FEDERAL
Santa Catarina

Visão geral sobre paradigmas de programação

Programação Orientada a Objetos (POO)

- Nasceu nos anos de 1960 (Simula 67), mas ganhou popularidade nos anos de 1980 com o Smalltalk.
- É um paradigma de programação diferente do que utilizamos até então, porém: o que se faz numa linguagem de PE se faz numa POO, mas não o contrário.
- O algoritmo é organizado em **classes** que servem como "modelos" para criar **objetos**.
- Funciona bem para programas pequenos, médios e grandes.
- Exemplo de linguagens de POO: Java, Kotlin, Python, C++, C#, etc.

INSTITUTO FEDERAL
Santa Catarina

Visão geral sobre paradigmas de programação

Entendendo a diferença entre PE e POO: programar é como brincar de massinha.



INSTITUTO FEDERAL
Santa Catarina

Visão geral sobre paradigmas de programação

Brincando de massinha: montando 3 carrinhos.

Na PE, se você quiser montar 3 carrinhos, você pega uma porção de massinha e modela um carinho. Depois, mais uma porção e modelo outro. Por fim, pega a última porção de massinha e faz o terceiro carrinho.

Ou você pode montar todas as rodinhas, todas as portas, depois todos os chassis e montar os carrinhos. Mas, você modelará cada uma das partes com suas mãos.

INSTITUTO FEDERAL
Santa Catarina

Visão geral sobre paradigmas de programação

Brincando de massinha: montando 3 carrinhos.

Na POO, você não modelará a massinha com as mãos. Você construirá uma forminha (uma classe) e, a partir dessa forminha, você fará quantos carrinhos (objetos) quiser.



INSTITUTO FEDERAL
Santa Catarina

Visão geral sobre paradigmas de programação

Brincando de massinha: montando 3 carrinhos.

E os carrinhos, saíram todos iguais? Não necessariamente, podemos variar a cor do carrinho, o tipo de aro, o tipo de motor (elétrico ou combustão), etc.



INSTITUTO FEDERAL
Santa Catarina

Introdução à Programação Orientada a Objetos (POO)

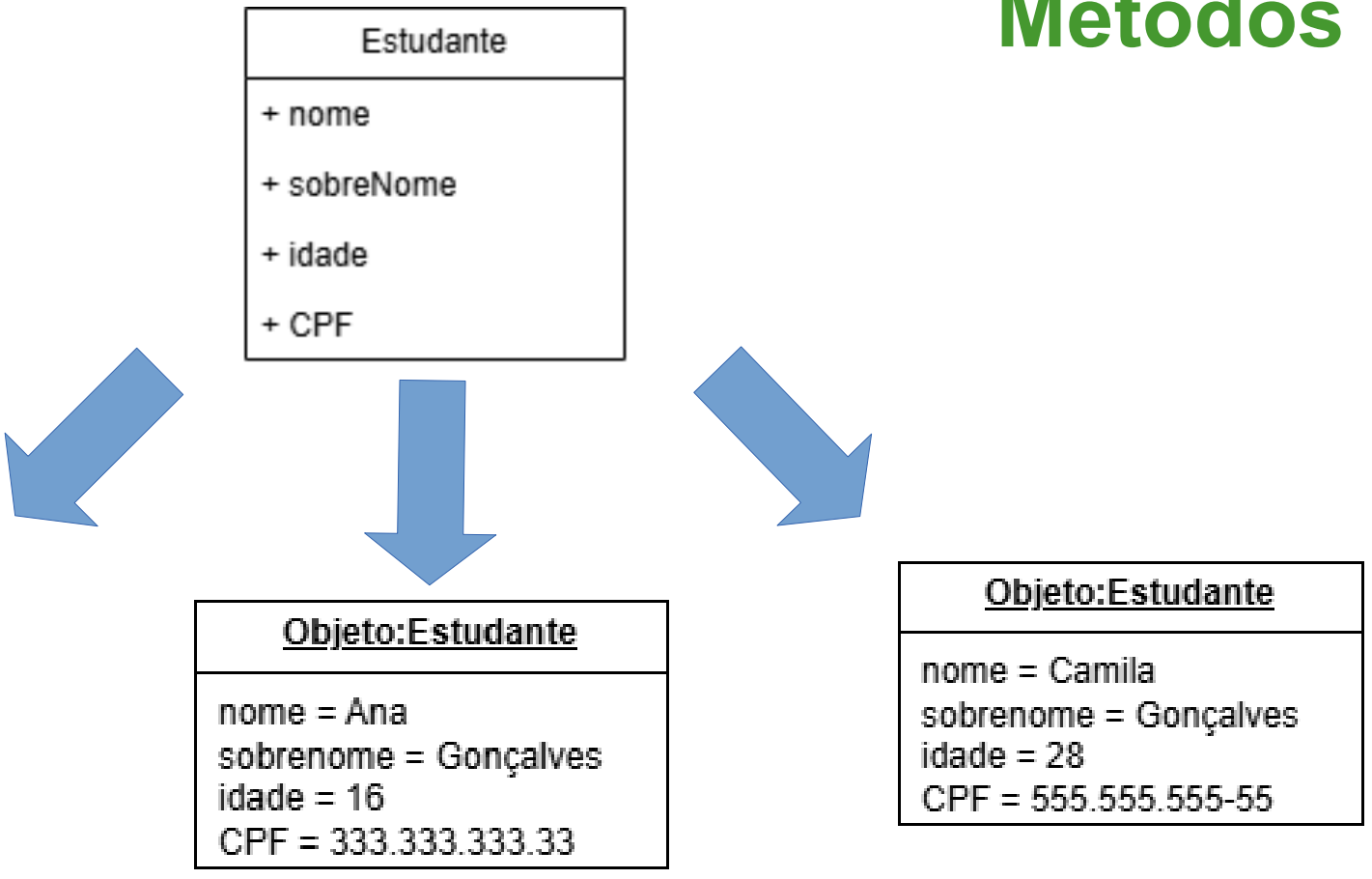
"Paradigma de programação baseado em ‘objetos’ que representam coisas do mundo real.

Objetos possuem atributos (dados) e métodos (comportamentos).”

Uma **classe** (forminha) serve para criar **objetos** e esses objetos possuem **atributos** e **métodos**.

Classes e Objetos + Atributos e Métodos

Exemplo prático:



INSTITUTO FEDERAL
Santa Catarina

Classes e Objetos + Atributos e Métodos

Exemplo prático:

Como eu implemento a classe Estudante em Python?

A classe Estudante possui 4 atributos (dados): nome, sobrenome, idade e CPF.

É interessante que criemos um método (“comportamento”, quase como uma “função”) construtor (chamado __init__) para inicializar os nossos atributos (dados).

INSTITUTO FEDERAL
Santa Catarina

Classes e Objetos + Atributos e Métodos

Classe Estudante (arquivo minhas_classes.py):

```
class Estudante:
    def __init__(self, nome, sobrenome, idade, cpf):
        self.nome = nome
        self.sobrenome = sobrenome
        self.idade = idade
        self.cpf = cpf
```

INSTITUTO FEDERAL
Santa Catarina

Classes e Objetos + Atributos e Métodos

Depois de definida a classe, como crio os objetos (programa.py)?

```
from minhas_classes import Estudante
# Criando 3 objetos da classe Estudante:
estudante1 = Estudante("Pedro", "da Silva", 17, "888.888.888-88")
estudante2 = Estudante("Ana", "Gonçalves", 16, "333.333.333-33")
estudante3 = Estudante("Camila", "Gonçalves", 28, "555.555.555-55")
```

INSTITUTO FEDERAL
Santa Catarina

Classes e Objetos + Atributos e Métodos

Depois de criar os objetos, como eu acesso seus atributos?

```
# como leio um atributo de um objeto?
print(estudante1.nome)

# como altero um atributo de um objeto?
estudante1.nome = "Ricardo"
```

INSTITUTO FEDERAL
Santa Catarina

Classes e Objetos + Atributos e Métodos

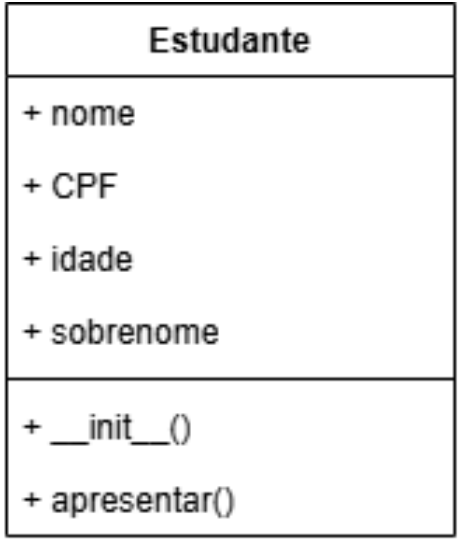
Exercício A: Altere o algoritmo para representar 3 colegas de turma. O algoritmo também deve, ao final, exibir o nome e a idade de cada um deles.



INSTITUTO FEDERAL
Santa Catarina

Métodos

Uma classe, além do método (comportamento) construtor (__init__), pode ter outros métodos auxiliares. Por exemplo, podemos criar um método na classe Estudante para que ele se apresente.



INSTITUTO FEDERAL
Santa Catarina

Métodos

Uma classe, além do método (comportamento) construtor (__init__), pode ter outros métodos auxiliares. Por exemplo, podemos criar um método na classe Estudante para que ele se apresente.

```
class Estudante:
    def __init__(self, nome, sobrenome, idade, cpf):
        self.nome = nome
        self.sobrenome = sobrenome
        self.idade = idade
        self.cpf = cpf

    def apresentar(self):
        return "Oi, meu nome é " + self.nome + " e tenho " + str(self.idade) + " anos."
```

INSTITUTO FEDERAL
Santa Catarina

Métodos

Como eu chamo o método e imprimo seu retorno?

```
# chamando o método e imprimindo seu resultado
print(estudante1.apresentar())
```

INSTITUTO FEDERAL
Santa Catarina

Métodos

Exercício B: Implemente a classe abaixo, crie 2 objetos e chame o método apresentar(). A apresentação deve mostrar o nome completo e a especialização do professor (Filosofia, Informática, etc).

Professor
+ nome + sobrenome + especializacao
+ __init__() + apresentar()



INSTITUTO FEDERAL
Santa Catarina

Exercícios

Resolva a Lista 06 dos exercícios 1 ao 5.



INSTITUTO FEDERAL
Santa Catarina

Listas de Objetos

Se estivermos trabalhando com poucos objetos, fica fácil de armazená-los em variáveis (um estudante, dois estudantes, etc).

E se estivermos trabalhando com uma quantidade indefinida de objetos (muitos estudantes)?

Para trabalhar com uma quantidade indefinida de objetos, podemos armazená-los em listas.

INSTITUTO FEDERAL
Santa Catarina

Listas de Objetos

Veremos algumas operações básicas com listas de objetos:

- Adicionando objetos dinamicamente;
- Exibindo os objetos da lista;
- Atualizando atributos;
- Removendo objetos da lista.

INSTITUTO FEDERAL
Santa Catarina

Listas de Objetos

Para vermos essas operações básicas, vamos tomar a classe abaixo como exemplo de partida:

```python
class Estudante:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def apresentar(self):
        return f"Sou {self.nome}, tenho {self.idade} anos."
```

INSTITUTO FEDERAL
Santa Catarina

Listas de Objetos

Criando a lista e adicionando objetos:

```
# Criando a lista vazia
estudantes = []

# Adicionando objetos dinamicamente
estudantes.append(Estudante("Carlos", 21))
estudantes.append(Estudante("Lucia", 23))
```

INSTITUTO FEDERAL
Santa Catarina

Listas de Objetos

Exibindo objetos da lista:

```
# Exibindo os estudantes
for estudante in estudantes:
    print(estudante.apresentar())
```

INSTITUTO FEDERAL
Santa Catarina

Listas de Objetos

Atualizando atributos de um objeto específico:

```
# Atualizando a idade de um estudante com nome Carlos
for estudante in estudantes:
    if estudante.nome == "Carlos":
        estudante.idade = 90
```

INSTITUTO FEDERAL
Santa Catarina

Listas de Objetos

Removendo objetos da lista:

```
# Removendo a estudante "Lucia"
for estudante in estudantes:
    if estudante.nome == "Lucia":
        estudantes.remove(estudante)
        break # não é necessário se desejar remover todas as "Lucias"
```

INSTITUTO FEDERAL
Santa Catarina

Exercícios

Exercício C: Utilizando como base os exemplos dos slides anteriores, faça alguns experimentos com listas de objetos adicionando, exibindo, atualizando atributos e excluindo colegas da sua turma.



INSTITUTO FEDERAL
Santa Catarina

Exercícios

Resolva, da Lista 06, os exercícios 6 ao 10.



INSTITUTO FEDERAL
Santa Catarina

Salvando e carregando objetos

É possível utilizar um módulo de **serialização** do Python para que os objetos possam ser salvos e carregados do disco. Veja exemplos de como salvar e como carregar nos próximos slides. Não esqueça de carregar o módulo no início do seu código:

```
import pickle
```

INSTITUTO FEDERAL
Santa Catarina

Salvando e carregando objetos

```
# Salvando lista de estudantes em um arquivo chamado dados.pkl
with open('dados.pkl', 'wb') as f:
    pickle.dump(estudantes, f)
    print("Objetos salvos com sucesso!")
```

INSTITUTO FEDERAL
Santa Catarina

Salvando e carregando objetos

```
# Carregando lista de estudantes de um arquivo chamado dados.pkl
with open('dados.pkl', 'rb') as f:
    estudantes = pickle.load(f)
    print("Objetos carregados com sucesso!")
```

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

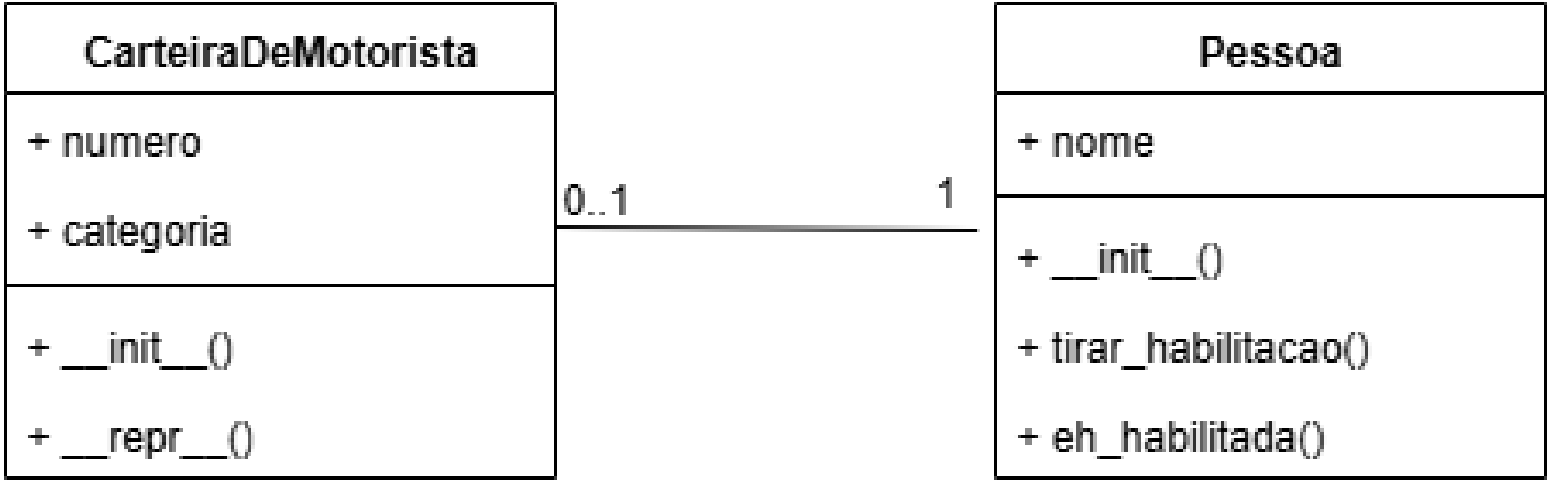
Conforme vimos no Modelo Conceitual, é possível definir alguns tipos de relacionamentos como 1..1, 1..* e *..*. Há diversas formas de implementar esses tipos de relacionamentos. Veremos, um das formas de implementação, para os seguintes relacionamentos:

- 0..1
- 1..1
- 1..n
- 0..n

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

Exemplo 0..1:



Uma pessoa pode ou não ter uma CarteiraDeMotorista. A CarteiraDeMotorista é a “parte”, a Pessoa é o “todo”.

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

```python
class CarteiraDeMotorista:
    def __init__(self, numero, categoria):
        self.numero = numero
        self.categoria = categoria

    def __repr__(self):
        # permite "printar" o objeto
        return f"<CNH nº {self.numero}, Cat: {self.categoria}>"
```

INSTITUTO FEDERAL
Santa Catarina

```python
class Pessoa:
    def __init__(self, nome):
        self.nome = nome
        # O relacionamento 0..1 começa como None (zero CNHs)
        self.cnh = None

    def tirar_habilitacao(self, numero, categoria):
        # Se a pessoa ainda não tem CNH (ainda é '0')
        if self.cnh is None:
            # O relacionamento vira '1'
            self.cnh = CarteiraDeMotorista(numero, categoria)
            return True
        else:
            return False # Falha, já possui CNH

    def eh_habilitada(self):
        if self.cnh is None:
            return False
        else:
            return True

    def __repr__(self):
        return f"<Pessoa: {self.nome}>"
```

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

```python
# Programa Principal - primeira parte
print("Criando o objeto 'Ana'...")
ana = Pessoa("Ana")

print(f"Objeto criado: {ana}")

# --- Verificando o estado inicial ('0') ---
print(f"A {ana.nome} é habilitada? {ana.eh_habilitada()}")
print(f"CNH da {ana.nome}: {ana.cnh}") # O atributo é None

# --- Mudando o estado para '1' ---
print(f"\n{ana.nome} foi tirar a habilitação...")
sucesso = ana.tirar_habilitacao("123456789", "B")
```

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

```
# Programa Principal - segunda parte
if sucesso:
    print("-> Sucesso! Habilitação obtida.")
else:
    print("-> Falha!")

# --- Verificando o novo estado ('1') ---
print(f"A {ana.nome} é habilitada? {ana.eh_habilitada()}")
print(f"CNH da {ana.nome}: {ana.cnh}") # Agora é um objeto
print(f"Número da CNH: {ana.cnh.numero}")
```

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

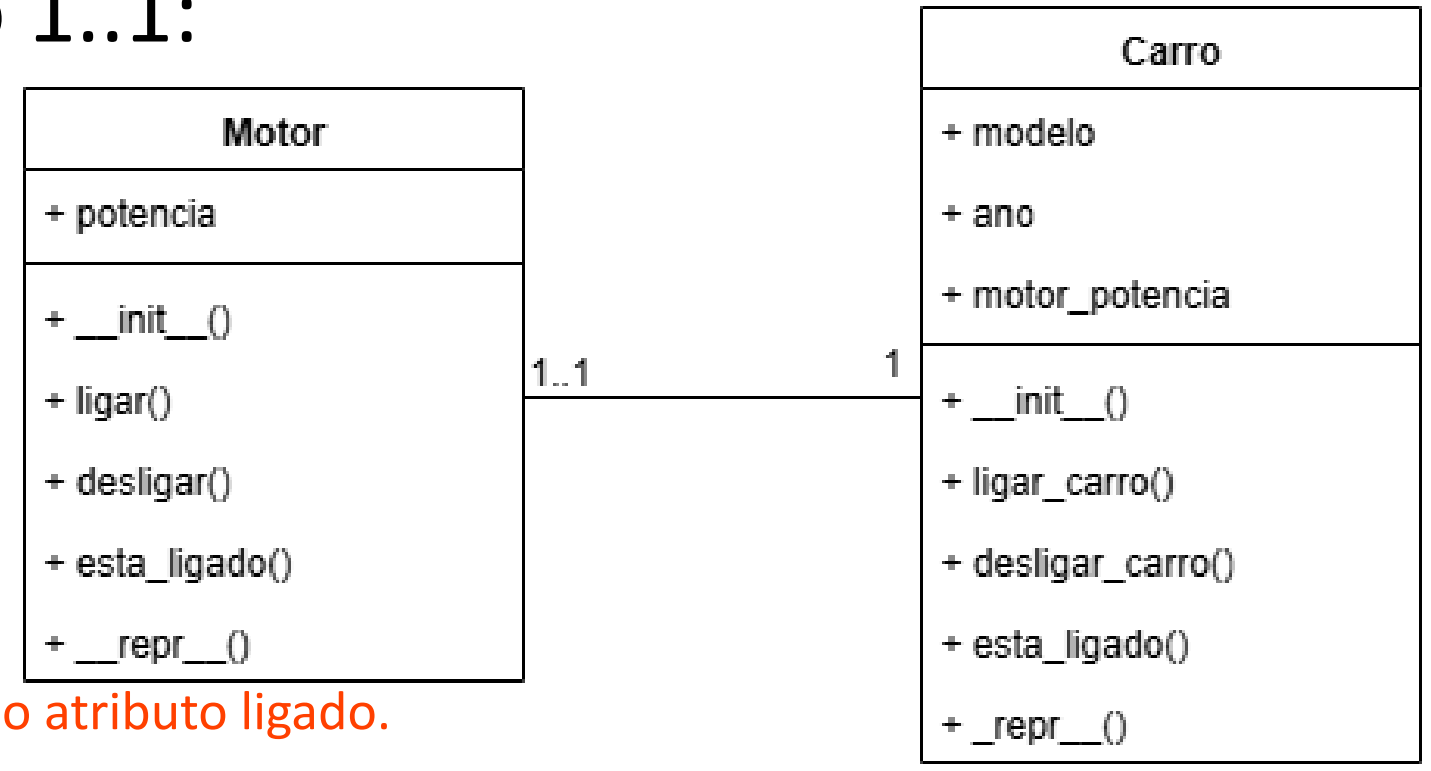
Exemplo 1..1:

- Um Carro (no nosso modelo) precisa ter exatamente um Motor para ser um carro funcional.
- O Carro (o "todo") é "fabricado" com o Motor (a "parte") já instalado.
- Um Carro e um Motor são coisas claramente distintas, com propriedades diferentes, e não podem ser "uma classe só".

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

Exemplo 1..1:



No Motor, faltou o atributo ligado.

INSTITUTO FEDERAL
Santa Catarina

```python
class Motor:
    def __init__(self, potencia_cv):
        self.potencia_cv = potencia_cv
        self.ligado = False

    def ligar(self):
        self.ligado = True

    def desligar(self):
        self.ligado = False

    def esta_ligado(self):
        return self.ligado

    def __repr__(self):
        return f"<Motor: {self.potencia_cv}cv>"
```

INSTITUTO FEDERAL
Santa Catarina

```python
class Carro:
    def __init__(self, modelo, ano, motor_potencia):
        self.modelo = modelo
        self.ano = ano
        # RELACIONAMENTO 1..1 (Composição)
        # O Motor é criado *obrigatoriamente* quando o Carro é criado.
        # O construtor do Carro 'pede' os dados para criar o Motor.
        # O atributo 'self.motor' NUNCA será None.
        self.motor = Motor(motor_potencia)

    def ligar_carro(self):
        self.motor.ligar()

    def desligar_carro(self):
        self.motor.desligar()

    def esta_ligado(self):
        # Expõe o estado do motor.
        return self.motor.esta_ligado()

    def __repr__(self):
        return f"<Carro: {self.modelo} {self.ano}>"
```

INSTITUTO FEDERAL
Santa Catarina

```python
# Programa Principal
# --- Criando o objeto 'todo' ---
print("Fabricando um novo carro (Fusca)...")

# Para criar o Carro, somos OBRIGADOS a passar os dados do Motor
# Isso 'força' o relacionamento 1..1 a existir desde o início.
meu_fusca = Carro("Fusca", 1975, 46)
print(f"Carro fabricado: {meu_fusca}")

# --- Verificando o estado (a ligação 1..1 JÁ EXISTE) ---
# É impossível ter o 'meu_fusca' sem o 'meu_fusca.motor'.
print(f"O motor dele é: {meu_fusca.motor}")

# --- Usando os objetos ---
print(f"\nO carro está ligado? {meu_fusca.esta_ligado()}")

print("Ligando o carro...")
meu_fusca.ligar_carro() # O 'Carro' pede para o 'Motor' ligar

print(f"O carro está ligado? {meu_fusca.esta_ligado()}")
print(f"O motor está ligado? {meu_fusca.motor.esta_ligado()}")
```

INSTITUTO FEDERAL
Santa Catarina

Exercícios

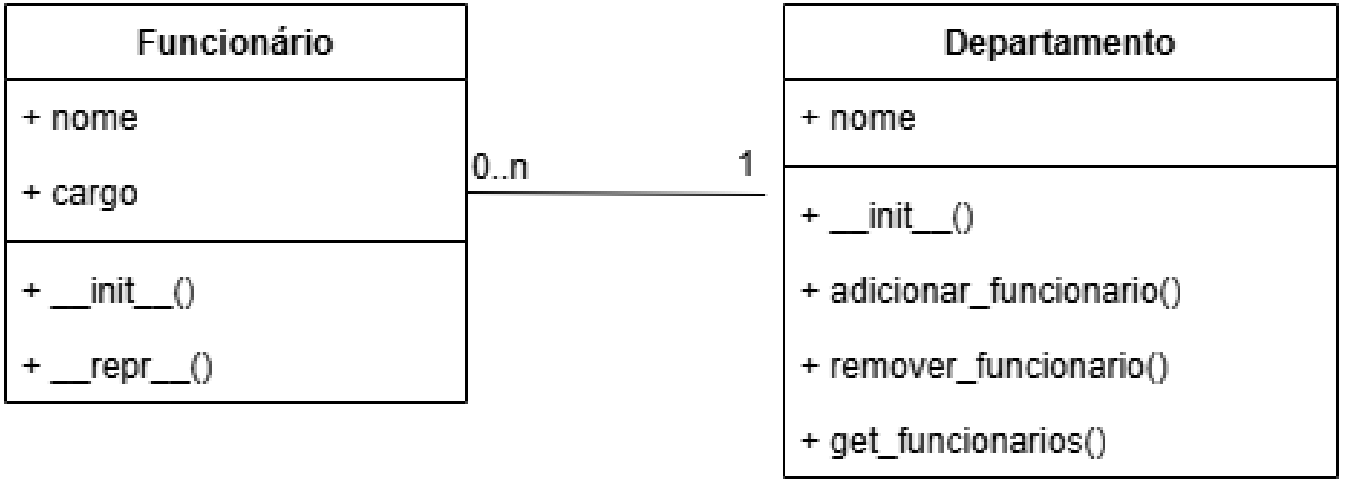
Resolva, da Lista 07, os exercícios 1 ao 4.



INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

Exemplo 0..n:



Um Departamento pode ter vários funcionários (ou nenhum) e um Funcionário sempre estará associado a um Departamento.

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

```python
class Funcionario:
    def __init__(self, nome, cargo):
        self.nome = nome
        self.cargo = cargo

    def __repr__(self):
        return f"Nome: {self.nome}, Cargo: {self.cargo}"
```

INSTITUTO FEDERAL
Santa Catarina

```python
class Departamento:
    def __init__(self, nome):
        self.nome = nome
        # A lista `funcionarios` implementa o relacionamento 0..n.
        # Um departamento começa com 0 funcionários (lista vazia).
        self.funcionarios = []

    def adicionar_funcionario(self, funcionario):
        self.funcionarios.append(funcionario)

    def remover_funcionario(self, nome_funcionario):
        # Itera sobre a lista de funcionários
        for func in self.funcionarios:
            if func.nome == nome_funcionario:
                # Usa o método remove() da lista para remover o objeto encontrado
                self.funcionarios.remove(func)
                return True
        # Se o loop terminar sem encontrar o nome
        return False

    def get_funcionarios(self):
        return self.funcionarios
```

```python
# --- Uso e Demonstração (Lógica de Apresentação no Programa Principal) ---
# 1. Criação de Instâncias de Funcionário
alice = Funcionario("Alice Rocha", "Gerente de Projetos")
bob = Funcionario("Bob Silva", "Analista Júnior")
carol = Funcionario("Carol Santos", "Desenvolvedora Sênior")

# 2. Criação de Instâncias de Departamento
marketing = Departamento("Marketing")
tecnologia = Departamento("Tecnologia")

# 3. Adicionando funcionários
tecnologia.adicionar_funcionario(alice)
print(f"-> {alice.nome} adicionado ao departamento de {tecnologia.nome}.")
tecnologia.adicionar_funcionario(bob)
print(f"-> {bob.nome} adicionado ao departamento de {tecnologia.nome}.")
tecnologia.adicionar_funcionario(carol)
print(f"-> {carol.nome} adicionado ao departamento de {tecnologia.nome}.")
```

INSTITUTO FEDERAL
Santa Catarina

os

```python
# 4. Removendo funcionários
# Removendo um funcionário existente
if tecnologia.remover_funcionario("Bob Silva"):
    print(f"Sucesso: Bob Silva foi removido.")
else:
    print(f"Falha: Bob Silva não encontrado.")
# Tentando remover um funcionário que não existe
if marketing.remover_funcionario("Maria Silva"):
    print(f"Sucesso: Maria Silva foi removida.")
else:
    print(f"Falha: Maria Silva não encontrada.")
```

INSTITUTO FEDERAL
Santa Catarina

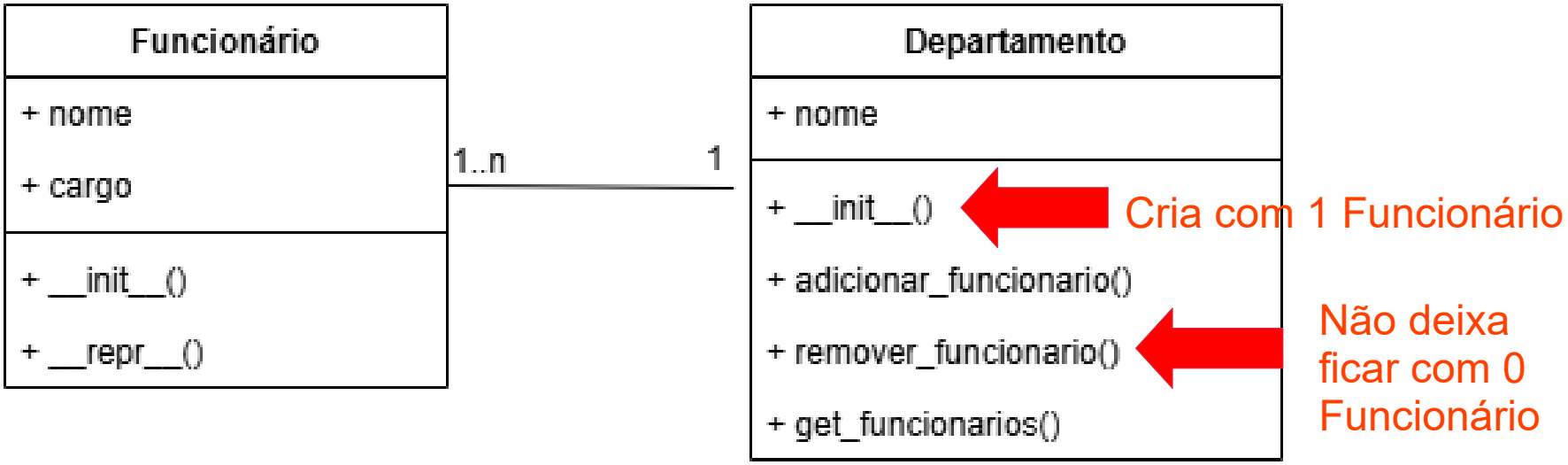
onamentos

```
# 5. Listando funcionários
# da Tecnologia
if len(tecnologia.get_funcionarios()) == 0:
    print("Tecnologia não tem funcionários")
else:
    print("Funcionários da Tecnologia:")
    for f in tecnologia.get_funcionarios():
        print(f)
# do Marketing
if len(marketing.get_funcionarios()) == 0:
    print("Marketing não tem funcionários")
else:
    print("Funcionários do Marketing:")
    for f in marketing.get_funcionarios():
        print(f)
```

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

Exemplo 1..n:



Mesmo exemplo, mudando apenas a cardinalidade.

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

As mudanças ficarão no método construtor (__init__) de Departamento, que exigirá um Funcionário:

```
def __init__(self, nome, primeiro_funcionario):
    self.nome = nome
    self.funcionarios = []
    # adiciona primeiro Funcionário ao Departamento,
    # garantindo o 1..n
    self.funcionarios.append(primeiro_funcionario)
```

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

E no momento de remover, sempre deve-se manter pelo menos 1 Funcionário no Departamento (código no próximo slide).

```python
def remover_funcionario(self, nome_funcionario):
    # 1. Encontra o objeto
    funcionario_a_remover = None
    for func in self.funcionarios:
        if func.nome == nome_funcionario:
            funcionario_a_remover = func
            break # Encontramos o primeiro, podemos parar de buscar
    # 2. Remove se encontrado
    if funcionario_a_remover:
        # Verifica se, ao remover, o departamento ficaria vazio (violando o 1..n)
        if len(self.funcionarios) > 1:
            self.funcionarios.remove(funcionario_a_remover)
            return True
        else:
            # Se for o último, o 1..n impede a remoção
            return "ÚLTIMO_FUNCIONARIO_IMPEDIDO"
    else:
        return False
```

INSTITUTO FEDERAL
Santa Catarina

Relacionamentos

No Programa Principal, devemos sempre passar o primeiro funcionário:

```
# 2. Criação de Instâncias de Departamento
tecnologia = Departamento("Tecnologia", alice)
marketing = Departamento("Marketing")
```

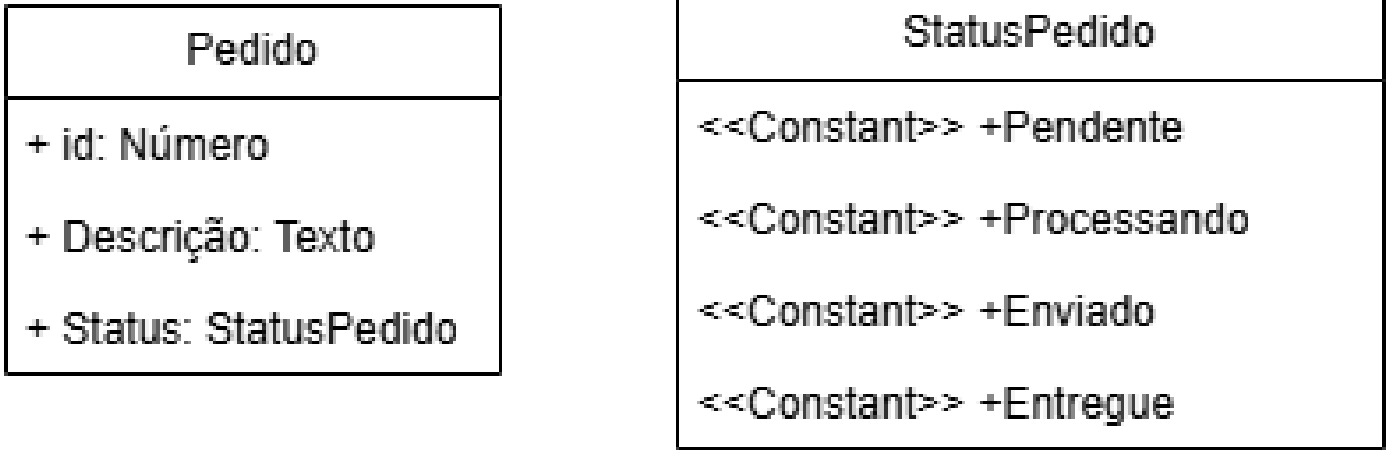
Funciona

Não funciona (erro)

INSTITUTO FEDERAL
Santa Catarina

Enumerações

Veremos, a seguir, como implementar os atributos do tipo enumeração:



INSTITUTO FEDERAL
Santa Catarina

Enumerações

Criando a enumeração:

```python
from enum import Enum

# Definição da enumeração
class StatusPedido(Enum):
    PENDENTE = "Pendente"
    PROCESSANDO = "Processando"
    ENVIADO = "Enviado"
    ENTREGUE = "Entregue"
```

INSTITUTO FEDERAL
Santa Catarina

Enumerações

Criando a classe Pedido (no próximo slide).

```python
# Classe que utiliza a enumeração
class Pedido:
    # Se o status do pedido não for definido no momento de instanciar o objeto,
    # por padrão, ele terá o status PENDENTE
    def __init__(self, id_pedido, descricao, status=StatusPedido.PENDENTE):
        self.id_pedido = id_pedido
        self.descricao = descricao
        self.status = status

    def atualizar_status(self, novo_status):
        # Se for definido um status inexistente...
        if not isinstance(novo_status, StatusPedido):
            # lança uma excessão
            raise ValueError("O status deve ser de uma das definições de StatusPedido.")
        # se não, atualiza o novo status
        self.status = novo_status

    # o método __str__ em Python permite que um objeto seja impresso com print
    def __str__(self):
        return f"Pedido {self.id_pedido}: {self.descricao} (Status: {self.status.value})"
```

INSTITUTO FEDERAL
Santa Catarina

Enumerações

Testando as classes:

```
# Exemplo de uso
pedido = Pedido(1, "Notebook")
print(pedido)

pedido.atualizar_status(StatusPedido.PROCESSANDO)
print(pedido)

pedido.atualizar_status(StatusPedido.ENTREGUE)
print(pedido)
```

INSTITUTO FEDERAL
Santa Catarina

Protegendo atributos

O Python, por padrão, não protege os atributos das classes.

O que isso quer dizer?

Quer dizer que, por exemplo, para nossa ContaBancaria, podemos mudar o saldo sempre que quisermos (sem sacar ou depositar).



INSTITUTO FEDERAL
Santa Catarina

Protegendo atributos

É possível “blindar” os atributos em Python?

Não! =~(

É sempre possível acessar os atributos através de algumas técnicas como o *name mangling*.

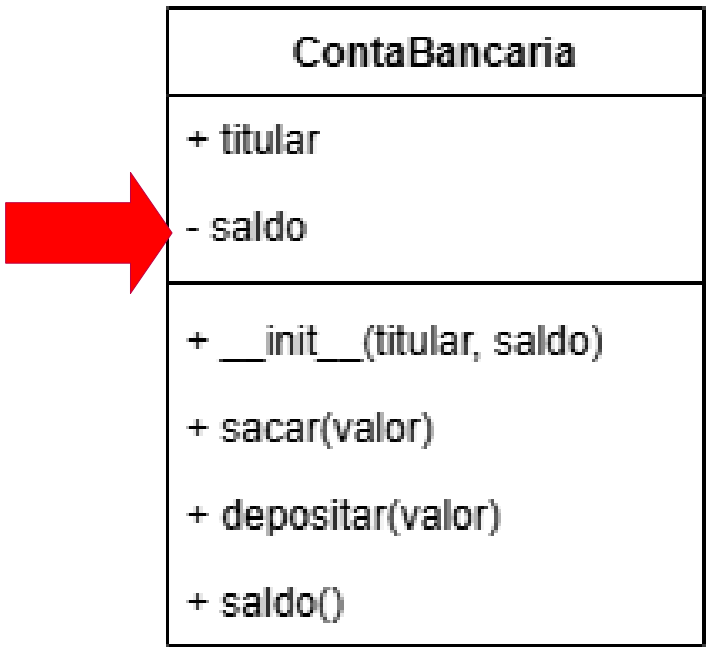
Porém, o “Python é uma linguagem projetada com a filosofia de ‘consulte o código, não oculte o código’. Em vez de proibir, ela sugere convenções para indicar intenções.”

Usaremos o @property para criar um Getter (“pegador”, método que retorna atributo) e o @atributo.setter para criar um Setter (“setador”, método que modifica um atributo).

INSTITUTO FEDERAL
Santa Catarina

Protegendo atributos

Veremos uma forma mais elegante de implementar o ContaBancaria:



INSTITUTO FEDERAL
Santa Catarina

Protegendo atributos

Veremos uma forma mais elegante de implementar o ContaBancaria:

```python
class ContaBancaria:
    def __init__(self, titular, saldo=0):
        self.titular = titular  # Atributo público
        self._saldo = saldo  # Atributo privado

    # Getter para saldo
    @property
    def saldo(self):
        return self._saldo

    # Setter para saldo (controle ao modificar)
    @saldo.setter
    def saldo(self, valor):
        print("Não é possível alterar o saldo diretamente!")
```

INSTITUTO FEDERAL
Santa Catarina

Protegendo atributos

Veremos uma forma mais elegante de implementar o ContaBancaria:

```
# Método para depositar dinheiro
def depositar(self, valor):
    if valor > 0:
        self._saldo += valor
    else:
        print("Valor de depósito deve ser positivo.")

# Método para sacar dinheiro
def sacar(self, valor):
    if valor > self._saldo:
        print("Saldo insuficiente!")
    elif valor > 0:
        self._saldo -= valor
    else:
        print("Valor de saque deve ser positivo.")
```

INSTITUTO FEDERAL
Santa Catarina

Protegendo atributos

Veremos uma forma mais elegante de implementar o ContaBancaria:

```
# Testando a classe
conta = ContaBancaria("João", 100)

# Acessando o titular (atributo público)
print(f"Titular: {conta.titular}")  # João

# Acessando e manipulando o saldo (via @property)
print(f"Saldo inicial: R$ {conta.saldo}")  # R$ 100
conta.depositar(50)
print(f"Saldo após depósito: R$ {conta.saldo}")  # R$ 150
conta.sacar(30)
print(f"Saldo após saque: R$ {conta.saldo}")  # R$ 120

# Tentando alterar saldo diretamente
conta.saldo = -10
print(f"Saldo final: R$ {conta.saldo}")
```

INSTITUTO FEDERAL
Santa Catarina

Protegendo atributos

Exercício D: Teste vários saques, depósitos e consultas de saldo na conta de João.



INSTITUTO FEDERAL
Santa Catarina

Referências

INSTITUTO FEDERAL
Santa Catarina

Duvidas?

